CSP352

AI-Augmented Software Development

LLMs as Software Components

Dhiman Saha
Amit Kumar Dhar

IIT Bhilai



Today

1. **Where we left off** – the Lab 2 bottleneck, and what Aider was doing about it.
2. **The request** – roles, and the object you actually send.
3. **The response** – what comes back, and how to parse it safely.
4. **Validation, conversation state, and retries** – the three things Aider was quietly handling for you.
5. **Live**: build a minimal script from scratch, one layer at a time.

Where we left off: the Lab 2 bottleneck

- In Lab 2, some sessions grew a conversation until the model's responses degraded or stalled – the **context window** filled up.
- In Lab 2b, you practiced the fix: **restart the session** / `/drop` **stale files** once context gets messy.
- Today: what `aider` was actually doing under the hood when that SOP kicked in – and why the fix was "restart", not "hope it recovers".

The question this lecture answers

What is a "context window", really – and what is your code responsible for once you're not talking to `aider` anymore?

Chatbot vs. raw API call

`aider` looked like a conversation. It wasn't one – it was your prompts, replayed:

- Assembling every message (yours, the model's, and system instructions) into one request, **every single call**.
- Tracking which files you'd shared, so it could resend their content too.
- Retrying automatically when a call failed or timed out.

Today, you write that layer yourself

Not to replace `aider` – to understand the mechanism it was quietly running, so the Lab 2b SOP stops being a rule you follow and becomes a consequence you can predict.

The request, as a software artifact

Not "how do you prompt well" (that was Week 2) – the concrete shape of the call your code makes:

```
response = client.chat.completions.create(
    model="main",
    messages=[
        {"role": "system", "content": "You are a support triage agent."},
        {"role": "user", "content": ticket_text},
    ],
)
```

- **Roles** (`system`/`user`/`assistant`) are how the request encodes "who said what" – this is the same structure `aider` was building for you every turn.
- `model` is a real, checkable value – not every key is valid for every account (we'll hit this live in a minute).

The response, as a software artifact

What actually comes back is an object, not a string:

```
ChatCompletion(id='chatcmpl-...',
  choices=[Choice(finish_reason='stop', index=0,
    message=ChatCompletionMessage(content='...', role='assistant'))],
  model='main', usage=CompletionUsage(completion_tokens=215,
    prompt_tokens=32, total_tokens=247))
```

- The text you want is buried several fields deep: `choices[0].message.content`.
- `usage` is where your token/cost accounting comes from – this is what grows every call once conversation state enters the picture.

Output parsing

```
category = response.choices[0].message.content.strip()
```

- The model's answer lives several fields deep in the response object – `choices[0].message.content`.
- Real responses are messier than the example: extra whitespace, punctuation, or an empty `choices` list.
- Parsing code that assumes a perfect reply is the single most common way an LLM-calling script breaks in production.

Schema-based validation (a first look)

- Parsing extracts a value; **validation** checks it's an allowed one before your code trusts it.
- Example: is `category` actually one of {`billing`, `technical`, `account`, `feature-request`, `complaint`}, or did the model answer in free text?
- Today: a light touch (a membership check). **Next week** (structured outputs): enforcing a schema on the request itself, so invalid shapes are rarer to begin with.

Conversation state: the illusion

There is no conversation

Every call resends the **entire history** – system prompt, every prior user message, every prior model reply. The model has no memory between calls; your client does the remembering, and pays for it every time.

- This is exactly what filled the context window in Lab 2: history kept growing, and every call got more expensive and more likely to hit the ceiling.
- "Restart" (Lab 2b's SOP) means exactly one thing at this layer: **throw away the history and start resending less**.

Error handling and retries

- Rate limits, transient network failures, and timeouts are **normal**, not exceptional – production code expects them.
- A retry without backoff just resends into the same rate limit. **Exponential backoff**: wait longer after each failed attempt, up to a retry limit.
- This is the layer `aider` was quietly running for you. Lab 3b builds it explicitly.

Before we build: what we just covered

- The **request**: roles, and a real object your code constructs.
- The **response**: a real object your code has to reach into, not text you can trust as-is.
- **Validation**: check the shape before you trust it.
- **Conversation state**: there is no conversation – only a growing history you resend.
- **Retries**: failures are normal; backoff makes retrying safe.

Now: watch all five become one script

Following along:
`lectures/week-03-llms-as-software-components/live-coding/`

Live, step 1: the bare call

```
client = OpenAI(
    base_url=os.environ["OPENAI_BASE_URL"],
    api_key=os.environ["OPENAI_API_KEY"],
)
response = client.chat.completions.create(
    model="main",
    messages=[
        {"role": "system", "content": "You are a support triage agent."},
        {"role": "user", "content": ticket_text},
    ],
)
print(response)
```

Watch for: the full response object – and how little of it we actually want.

Live, step 2: parse and validate

```
if not response.choices:
    raise RuntimeError("empty choices in response")
category = response.choices[0].message.content.strip().lower()
if category not in ALLOWED_CATEGORIES:
    raise ValueError(f"unrecognized category: {category!r}")
```

Watch for: what changes in the `system` prompt to make parsing reliable in the first place.

Live, step 3: conversation state

```
history = [{"role": "system", "content": SYSTEM_PROMPT}]
for ticket_text in tickets:
    history.append({"role": "user", "content": ticket_text})
    response = client.chat.completions.create(
        model="main", messages=history)
    category = response.choices[0].message.content.strip().lower()
    history.append({"role": "assistant", "content": category})
    print(f"{category} (history now has {len(history)} messages)")
```

Watch for: `len(history)` growing every ticket – this is the Lab 2 bottleneck, live.

Live, step 4: retries

```
for attempt in range(1, MAX_ATTEMPTS + 1):
    try:
        return client.chat.completions.create(
            model="main", messages=messages)
    except (RateLimitError, APIError):
        if attempt == MAX_ATTEMPTS:
            raise
        time.sleep(2 ** attempt)
```

Watch for: the wait growing each retry (2, 4, 8…) – not hammering the same limit.